PROJECT REPORT

On

MINI WEATHER STATION

SUBMITTED BY:

ANOOF MUHAMMED K

BHAGATH TP

DANIA ABDULLA

GAUTHAM LASER

IVANA ANTO

BACHELOR OF TECHNOLOGY in

ELECTRONICS AND COMMUNICATION



SCHOOL OF ENGINEERING

COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

KOCHI-682022

SEPTEMBER 2026

1. Abstract

This project presents a **Mini Weather Station** that measures temperature and atmospheric pressure, estimates altitude, and gives an early warning of rain or storms from the way the pressure changes over time. The system is built from two cooperating microcontrollers: an **ESP32-based MYOSA motherboard** that handles sensing and display, and an **8051 microcontroller (AT89C52)** that acts as a dedicated alarm unit.

The MYOSA board reads a **BMP180** digital barometric sensor over the I2C bus every two seconds. From each reading it calculates the altitude using the international barometric formula, keeps the maximum and minimum values of temperature and pressure, and stores one pressure sample every ten minutes in a three-hour history. Comparing the present pressure with the value three hours earlier gives the **pressure tendency**, the same quantity that meteorologists use for short-range forecasting. A fall of 1 hPa or more in three hours is classified as "rain likely", a fall of 3 hPa or more as a "storm warning", and a rise of 1 hPa or more as "improving". All readings are shown on a 128 × 64 **OLED display**.

The weather status is sent every two seconds as a short serial message (for example `#R`) at 9600 baud to the 8051. The 8051 receives it using its built-in UART and serial interrupt, and drives a warning LED and a buzzer: steady light for rain, blinking light with a 10-second buzzer alarm for a storm, and a fast blink if messages stop arriving for five seconds (link failure). The 8051 part was simulated in **Proteus 8** and connected live to the real MYOSA hardware through the laptop's USB serial port using the COMPIM component.

The firmware was written in VS Code, using PlatformIO (Arduino framework) for the ESP32 and the free SDCC compiler for the 8051. Testing showed correct sensor readings (about 30.6 °C and 1010.7 hPa indoors at Kochi), correct classification in a built-in demonstration mode that simulates a falling barometer, and correct alarm behaviour on the 8051. The project demonstrates I2C sensor interfacing, UART communication between two different microcontroller families, timer- and interrupt-driven programming of the 8051, and simple data processing for weather prediction, all at a low cost.

2. Introduction

Weather affects agriculture, transport, construction, outdoor events and daily life. Large weather stations use many expensive instruments, but one of the most useful measurements, **atmospheric pressure**, can be made with a small, low-cost digital sensor. Pressure is closely linked to the weather: when a low-pressure system approaches, the pressure at a place falls, and clouds, rain and wind usually follow. When high pressure builds, the weather generally becomes settled. Long before electronic sensors existed, this idea was used in the household barometer marked "Rain", "Change" and "Fair".

A barometer reading taken at one moment says little by itself, because normal pressure depends on the altitude of the place and varies from day to day. What matters for forecasting is the **rate of change** of pressure. Meteorological services report this as the pressure tendency over three hours. A slow, steady fall suggests worsening weather, while a rapid fall of several hectopascals in three hours often comes before strong winds and storms. A microcontroller is well suited to this task because it can take readings continuously, remember past values and compare them automatically.

This project builds such a system. The sensing, calculation and display work is done by the ESP32 microcontroller on the **MYOSA** (Make Your Own Sensors Applications) kit, a plug-and-play sensor learning platform from the IEEE Sensors Council. The warning output is handled by an **8051 microcontroller**, the classic 8-bit microcontroller studied in our Embedded Systems course. Using the two together shows how a modern controller and a classic one can share work through a simple serial protocol, which is common in real products where a main processor talks to smaller helper controllers.

The rest of this report is organised as follows. Chapters 3 and 4 state the problem and objectives. Chapter 5 explains the theory behind the design. Chapters 6 to 8 describe the system architecture, the hardware and the circuit. Chapter 9 covers the software design of both controllers, and Chapter 10 gives the algorithms. Chapters 11 and 12 describe how the system was implemented, tested and debugged. The remaining chapters cover cost, applications, merits and demerits, future scope and the conclusion. The complete source code is given in the appendices.

3. Problem Statement

People who work outdoors, farmers and small institutions often have no simple local tool that warns them when the weather is about to change. Reading a barometer by hand and remembering earlier values is inconvenient and easily forgotten, and online forecasts cover large areas rather than the exact location. There is a need for a **low-cost, self-contained device** that:

- continuously measures local temperature and atmospheric pressure,
- automatically tracks how the pressure changes over a three-hour period,
- shows the readings and a simple forecast on a display, and

- raises a clear visual and audible alarm when rain or a storm becomes likely, and also signals when its own data link has failed.

The course also requires that the 8051 microcontroller be part of the design. The challenge is therefore to divide the work sensibly between the 8051 and the sensor platform, and to make the two communicate reliably.

4. Objectives

- To interface the BMP180 digital pressure and temperature sensor and an SSD1306 OLED display with the ESP32 on the MYOSA board using the I2C bus.
- To calculate altitude from pressure and to record the maximum and minimum temperature and pressure.
- To store a three-hour pressure history and classify the weather as stable, improving, rain likely or storm warning from the pressure tendency.
- To design a simple, robust serial message format and send the weather status to the 8051 at 9600 baud.
- To program the 8051 (AT89C52) in embedded C using its UART, Timer 0, Timer 1 and serial interrupt, and to drive a warning LED and buzzer.
- To detect a broken communication link on the 8051 side using a software timeout.
- To simulate the 8051 alarm unit in Proteus and link it in real time to the physical MYOSA hardware.
- To provide a demonstration mode so that the warnings can be shown without waiting hours for a real change in weather.

5. Background Theory

5.1 Atmospheric pressure and weather

Atmospheric pressure is the weight of the air above a given point, measured in hectopascals (hPa); 1 hPa equals 100 Pa or 1 millibar. The average pressure at sea level is 1013.25 hPa. Areas of low pressure have rising air, which cools and forms clouds and rain. Areas of high pressure have sinking air and usually clear skies. When a low-pressure system moves towards a place, the local pressure falls, so a falling barometer is an early sign of bad weather.

Weather services describe the three-hour change of pressure, called the **pressure tendency**, with standard terms. Table 1 shows the commonly used categories. Our system uses two simplified thresholds based on these: a fall of at least 1.0 hPa means rain is likely, and a fall of at least 3.0 hPa means a storm warning.

Table 1: Pressure tendency categories and the simplified status levels used

Change in 3 hours	Description	Status used in this project
less than 0.1 hPa	Steady	S – Stable
0.1 to 1.5 hPa	Rising / falling slowly	S (below 1.0 hPa), R (fall of 1.0 hPa or more)
1.6 to 3.5 hPa	Rising / falling	R – Rain likely (fall), I – Improving (rise)
3.6 to 6.0 hPa	Rising / falling quickly	W – Storm warning (fall of 3.0 hPa or more)
more than 6.0 hPa	Rising / falling very rapidly	W – Storm warning

5.2 Calculating altitude from pressure

Pressure falls as height increases, by roughly 1 hPa for every 8 m near sea level. The international barometric formula (also given in the BMP180 datasheet) converts a pressure reading P into an approximate altitude h:

$$h = 44330 \times [\, 1 - (P / P_0)^{(1/5.255)} \,] \ \text{metres}, \quad \text{where } P_0 = 1013.25 \text{ hPa}$$

Here 1/5.255 ≈ 0.1903. Because the formula assumes standard sea-level pressure, the result is only an estimate: if the actual sea-level pressure on a given day is different, the calculated altitude shifts by several metres. This is why the altitude shown by the station drifts slightly as the weather changes.

5.3 The I2C bus

I2C (Inter-Integrated Circuit) is a two-wire serial bus. **SDA** carries data and **SCL** carries the clock. Both lines are open-drain with pull-up resistors, so several devices can share the same two wires. The ESP32 acts as the master and each sensor is a slave with a fixed 7-bit address: the BMP180 uses **0x77** and the OLED display uses **0x3C**. The master starts every transfer by sending the address, so only the addressed device responds. This allows the sensor and the display to be connected in parallel on GPIO 21 (SDA) and GPIO 22 (SCL).

5.4 UART serial communication

A UART (Universal Asynchronous Receiver/Transmitter) sends data one bit at a time without a separate clock line. Each byte is framed by a start bit (logic 0), eight data bits (least significant

bit first) and a stop bit (logic 1); this format is called **8N1**. Both ends must agree on the bit rate, the **baud rate**; this project uses 9600 baud, so each bit lasts about 104 µs and each byte about 1.04 ms. The transmitting pin (TXD) of one device connects to the receiving pin (RXD) of the other, and both share a common ground.

5.5 8051 timers, baud-rate generation and interrupts

The 8051 has two 16-bit timers (the 8052 family, including the AT89C52, has a third). With the standard 12-clock core, each timer increments once per **machine cycle**, which is 12 oscillator periods. With an 11.0592 MHz crystal:

$$\textit{Machine cycle} = 12 / 11.0592\ \text{MHz} = 1.085\ \mu s$$

Baud rate (Timer 1). In serial mode 1 the baud rate comes from Timer 1 working in mode 2 (8-bit auto-reload). With SMOD = 0:

$$\textit{Baud rate} = f(\text{osc}) / [384 \times (256 - TH1)]$$

$$TH1 = 256 - 11\,059\,200 / (384 \times 9600) = 256 - 3 = 253 = FDh$$

The crystal value 11.0592 MHz is chosen because it divides exactly into standard baud rates. With a 12 MHz crystal the same calculation gives 3.26, which cannot be loaded exactly; using TH1 = FDh would give about 10 417 baud, an error of 8.5 %, which is far too large for reliable reception. This matters in practice: Proteus sets the AT89C52 clock to 12 MHz by default, so it had to be changed to 11.0592 MHz in the component properties.

10 ms time base (Timer 0). Timer 0 is used in mode 1 (16-bit). For a 10 ms tick:

$$\textit{Counts} = 10\ \text{ms} / 1.085\ \mu s = 9216 \quad \Rightarrow \quad \textit{reload} = 65\,536 - 9216 = 56\,320 = DC00h$$

Interrupts. The 8051 can interrupt the main program when an event occurs. This project uses the **Timer 0 overflow interrupt** (vector 000Bh), which adds one to a tick counter every 10 ms, and the **serial interrupt** (vector 0023h), which runs whenever a byte has been received (RI = 1). The interrupts are enabled through the IE register (ET0, ES and the global enable EA).

6. System Overview and Block Diagram

The system is divided into a sensing unit and an alarm unit, as shown in Figure 1.

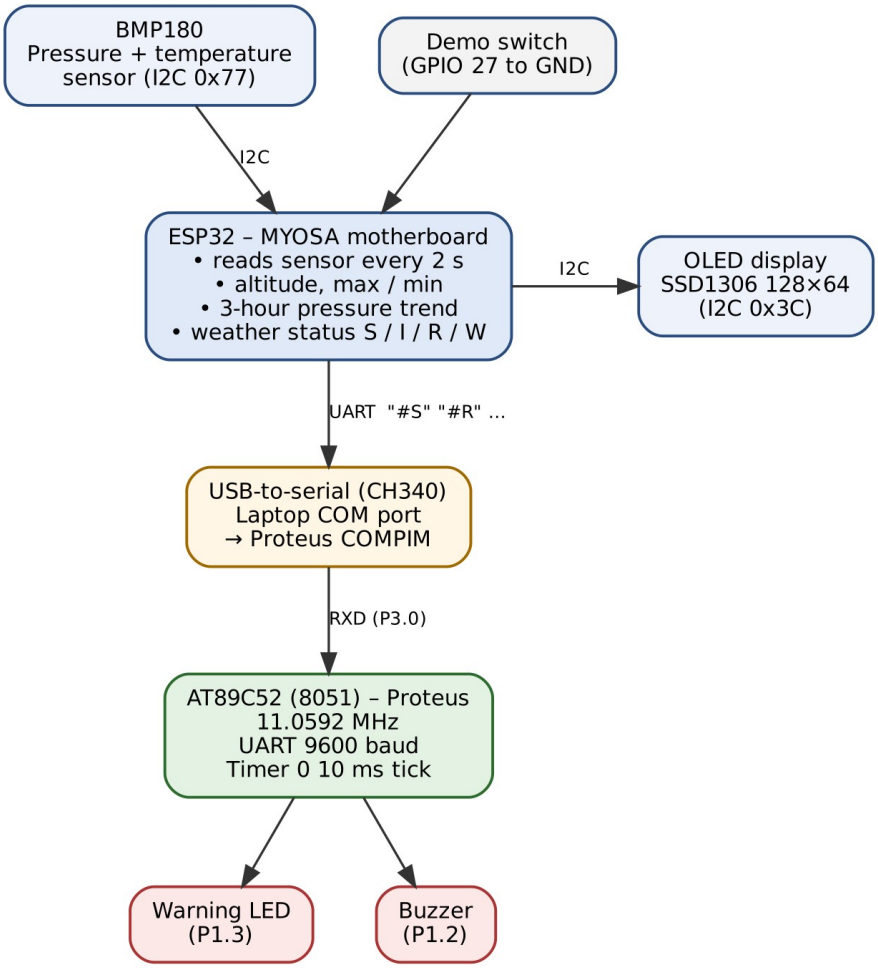


Figure 1: Block diagram of the Mini Weather Station

Sensing unit (real hardware). The BMP180 sensor and the OLED display are connected to the ESP32 on the MYOSA motherboard through the I2C bus. The ESP32 reads the sensor, performs all calculations, shows the results on the OLED, and decides the weather status. A jumper on GPIO 27 switches on the demonstration mode.

Communication link. The status is sent as a two-character message (`#S`, `#I`, `#R` or `#W`) followed by a newline, every two seconds, at 9600 baud 8N1. The message leaves the ESP32 through its USB port (CH340 USB-to-serial converter), reaches the laptop as COM7, and is passed into the Proteus simulation by the **COMPIM** (COM port physical interface model) component, which connects to the RXD pin of the simulated 8051. The same message is also sent on GPIO 25 (UART 2), so a real 8051 board could be connected directly in future.

Alarm unit (Proteus simulation). The AT89C52 receives the status in its serial interrupt and controls a red warning LED on P1.3 and a buzzer on P1.2. It also watches for missing messages and signals a link failure.

Division of work. The ESP32 does the floating-point maths, the I2C communication and the graphics, which would be slow and memory-hungry on an 8051 (it has only 256 bytes of

internal RAM). The 8051 does what it is best at: fast, predictable, interrupt-driven control of output devices. This division also means the alarm keeps working correctly even if the display code changes, because the two parts only share a very small, well-defined message format.

7. Hardware Description

7.1 Components used

Table 2: Hardware components

Component	Specification	Function in the project
MYOSA motherboard	ESP32 dual-core 32-bit MCU, 3.3 V logic, Wi-Fi/Bluetooth, CH340 USB-serial	Main controller: sensing, calculation, display, sends status
BMP180 sensor board	Bosch digital barometer, 300–1100 hPa, with temperature sensor, I2C address 0x77	Measures pressure and temperature
OLED display board	SSD1306 driver, 128 × 64 pixels, monochrome, I2C address 0x3C	Shows readings, max/min and forecast
AT89C52 (8051)	8-bit MCU, 8 KB flash, 256 B RAM, 32 I/O lines, 3 timers, full-duplex UART	Alarm unit: receives status, drives LED and buzzer
Crystal + 2 × 22 pF	11.0592 MHz	Clock for the 8051 (exact 9600 baud)
Reset network	10 µF capacitor, 10 kΩ resistor	Power-on reset of the 8051
Red LED + resistor	LED with 330 Ω series resistor	Visual warning (P1.3)
Buzzer	5 V	Audible storm alarm (P1.2)
Jumper wire	GPIO 27 to GND	Switches on demonstration mode
USB cable	Micro-USB	Power, programming and serial link to the laptop

7.2 MYOSA motherboard (ESP32)

The MYOSA motherboard is built around the ESP32, a 32-bit dual-core microcontroller with built-in Wi-Fi and Bluetooth. It works at 3.3 V and provides I2C connectors into which the MYOSA sensor boards plug directly. An on-board CH340 USB-to-serial converter lets the laptop program the ESP32 and exchange serial data with it; on our laptop it appears as **COM7**. The board is programmed through the Arduino framework.

7.3 BMP180 barometric pressure sensor

The BMP180 from Bosch is a digital barometric pressure sensor with an integrated temperature sensor. It measures 300 to 1100 hPa, which covers all weather conditions from about 500 m below sea level to about 9000 m above it. Each chip is factory-calibrated, and the calibration coefficients are stored in its internal memory. The ESP32 reads these once and uses them to turn the raw readings into temperature in °C and pressure in pascals. We used the Adafruit BMP085/BMP180 library in its default ultra-high-resolution mode. The sensor works from a 3.3 V supply and communicates over I2C at address 0x77.

7.4 SSD1306 OLED display

The display is a 0.96-inch, 128 × 64 pixel monochrome OLED driven by the SSD1306 controller. OLED pixels produce their own light, so no backlight is needed and the display is readable in the dark. The controller has an internal charge pump that generates the panel voltage from 3.3 V. It is connected on the same I2C bus at address 0x3C and is drawn using the Adafruit SSD1306 and GFX libraries.

7.5 AT89C52 microcontroller (8051 family)

The AT89C52 is an 8051-compatible microcontroller with 8 KB of on-chip flash program memory, 256 bytes of RAM, 32 programmable I/O lines in four ports, three 16-bit timer/counters, a full-duplex serial port and several interrupt sources. Pin 31 (EA) is tied to +5 V so that the program runs from internal flash. A port pin of the 8051 can sink much more current than it can source, so the LED and buzzer are connected between +5 V and the port pin. Writing **0** to the pin turns the device **on** (active-low output).

Table 3: 8051 pin allocation

8051 pin	Name	Connected to	Purpose
10	P3.0 / RXD	COMPIM RXD (ESP32 data)	Serial input, 9600 baud
4	P1.3	LED cathode (anode → 330 Ω → +5 V)	Warning LED, active-low
3	P1.2	Buzzer (other end → +5 V)	Storm buzzer, active-low
18, 19	XTAL2, XTAL1	11.0592 MHz crystal, 22 pF to GND	Clock
9	RST	10 µF to +5 V, 10 kΩ to GND	Power-on reset
31	EA	+5 V	Use internal program memory
40, 20	VCC, GND	+5 V, ground	Power supply

Table 4: ESP32 (MYOSA) pin allocation

ESP32 pin	Connected to	Purpose
GPIO 21 (SDA)	BMP180 SDA, OLED SDA	I2C data
GPIO 22 (SCL)	BMP180 SCL, OLED SCL	I2C clock
3V3, GND	VCC and GND of both boards	Power for sensor and display
GPIO 27	Jumper to GND (optional)	Demo mode input (internal pull-up)
USB (UART 0)	Laptop COM7 → Proteus COMPIM	Status messages, debug output, programming
GPIO 25 (UART 2 TX)	Not used (reserved)	Status for a real 8051 board

8. Circuit Diagram

Figure 2 shows the connections on the MYOSA side. The BMP180 and OLED share the same four wires (3.3 V, GND, SDA and SCL). The MYOSA sensor boards plug straight into the motherboard's I2C connectors, so no external pull-up resistors or breadboard were needed.

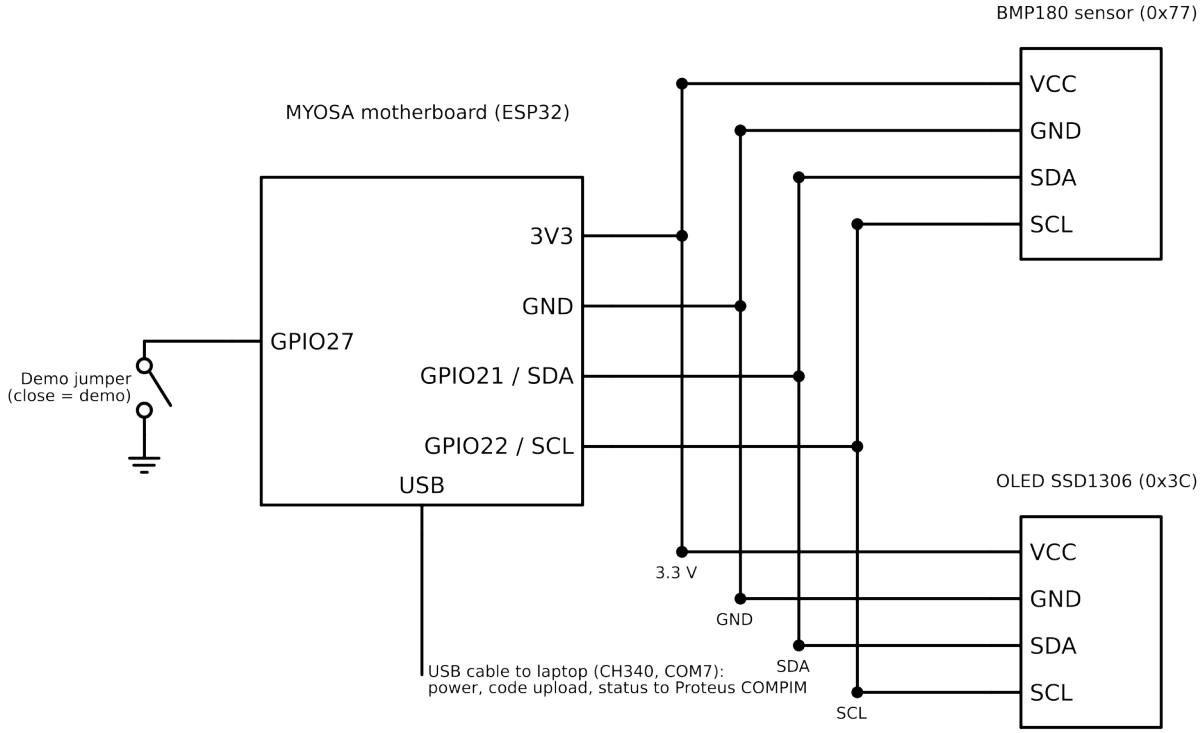


Figure 2: Wiring of the sensing unit (MYOSA motherboard, BMP180 and OLED)

Figure 3 shows the complete 8051 alarm circuit. The crystal, its two 22 pF capacitors and the reset network are needed when the circuit is built on real hardware. In Proteus the AT89C52 model takes its clock frequency from the component properties, so these parts may be left out of the simulation.

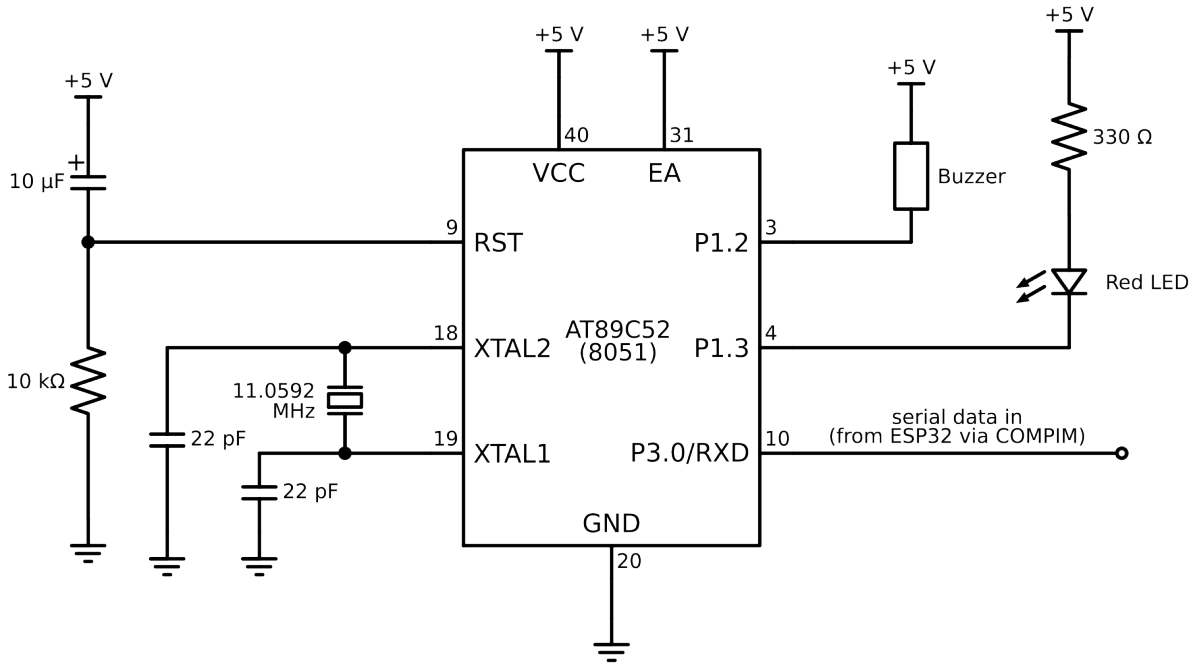


Figure 3: Circuit diagram of the 8051 alarm unit

Figure 4 shows the circuit as drawn in Proteus 8. During unit testing, a Virtual Terminal was connected to P3.0/RXD in place of the COMPIM block so that test messages could be typed by hand. In the final setup the COMPIM block, set to COM7 at 9600 baud, feeds the live data from the MYOSA board into the same pin.

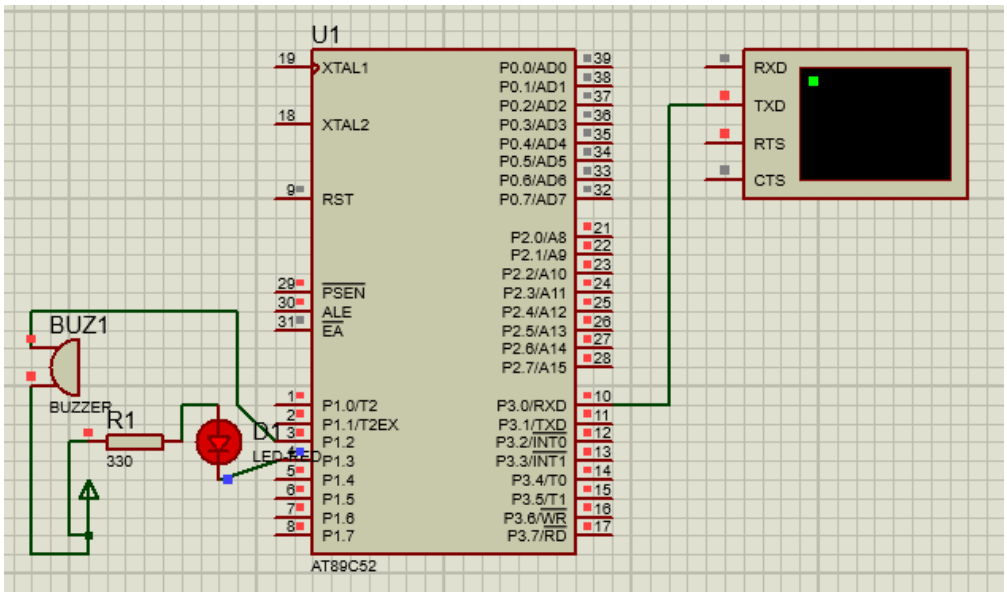


Figure 4: Proteus schematic of the alarm unit during simulation (red/blue squares show live logic levels)

Note for a hardware build: the buzzer is connected directly to the port pin, which is acceptable in simulation and for very small piezo buzzers. A physical buzzer that needs more than a few milliamperes should be driven through a transistor (for example a BC557 PNP with a 1 kΩ base resistor).

9. Software Design

9.1 Development tools

Table 5: Software tools

Tool	Version / detail	Used for
Visual Studio Code	with PlatformIO IDE extension	Editing and building both programs
PlatformIO	espressif32 platform, Arduino framework, board esp32dev	Compiling and uploading the ESP32 code; serial monitor
Libraries	Adafruit BMP085, Adafruit SSD1306, Adafruit GFX	BMP180 driver, OLED driver and graphics
SDCC	Small Device C Compiler 4.6.0, target mcs51	Compiling the 8051 C code to an Intel HEX file
Proteus 8 Professional	ISIS 8.13, AT89C52 model, COMPIM, Virtual Terminal	Simulating the 8051 alarm unit
VS Code build task	tasks.json (Ctrl+Shift+B)	One-key build of the 8051 HEX file

Both programs are kept in a single project folder: `src/main.cpp` for the ESP32, `8051/weather_8051.c` for the 8051, `platformio.ini` for the ESP32 build settings and `.vscode/tasks.json` for the 8051 build command. The 8051 build produces `8051/build/weather_8051.hex`, which is loaded into the AT89C52 in Proteus.

9.2 Communication protocol

The protocol was kept very simple so that the 8051 can decode it with almost no memory. Each message has three bytes: a start marker #, one status letter, and a newline character. The ESP32 also prints human-readable debug lines on the same USB port, for example `T=30.6 P=1010.7 A=21 drop=0.0 state=S`. These contain letters such as S and R too, so the 8051 accepts a status letter **only if it comes immediately after a** #. Every other character is ignored.

Table 6: Status messages and alarm actions

Message	Meaning	Condition (pressure change in 3 h)	8051 output
#S	Stable	between −1.0 and +1.0 hPa	LED off, buzzer off
#I	Improving	rise of 1.0 hPa or more	LED off, buzzer off
#R	Rain likely	fall of 1.0 hPa or more	LED steady on
#W	Storm warning	fall of 3.0 hPa or more	LED blinks (0.5 s), buzzer beeps for 10 s
(none for 5 s)	Link lost	–	LED blinks fast (0.1 s)

9.3 ESP32 firmware

The ESP32 program (Appendix A) is organised into small functions:

- **setup()** starts the USB serial port at 9600 baud and UART 2 on GPIO 25, enables the internal pull-up on GPIO 27, starts the I2C bus on GPIO 21/22, scans the bus and prints the addresses found, and initialises the BMP180 and the OLED.

- **updateWeather()** reads temperature and pressure, applies the demo offset if the jumper is fitted, calculates altitude, updates the maximum and minimum values, stores a pressure sample every 10 minutes, calculates the pressure drop over three hours and decides the status.
- **drawPage()** draws one of three OLED pages: Live (temperature, pressure, altitude), Max/Min, and Forecast (three-hour change and status in large text).
- **loop()** uses `millis()` timers instead of `delay()`. It reads the sensor every 2 s and sends the status, and changes the OLED page every 4 s. When a rain or storm warning starts, it jumps straight to the Forecast page and then alternates between Live and Forecast.

Three-hour pressure history (ring buffer)

Storing a reading every 2 s for three hours would need 5400 values. Instead the program stores one value every 10 minutes in an array of 18 elements (18 × 10 min = 3 h). An index variable points to the next position to overwrite and wraps from 17 back to 0. After each write, `slot` points to the **oldest** stored value, so the pressure drop is simply:

$$drop = hist[slot] - P(now)$$

A positive drop means the pressure has fallen. When the program starts, all 18 slots are filled with the first reading, so the drop starts at zero and builds up as real data arrives.

Demonstration mode

Real pressure changes of 1–3 hPa take hours, which is impractical for a viva. When GPIO 27 is connected to ground, the program subtracts an offset from the measured pressure that grows by 0.1 hPa at every 2-second reading. The drop therefore reaches 1.0 hPa after about 20 s (RAIN LIKELY) and 3.0 hPa after about 60 s (STORM WARNING). Removing the jumper resets the offset to zero, and the readings return to normal at once.

9.4 8051 firmware

The 8051 program (Appendix B) is written in C for SDCC. It uses the special function registers listed in the table below.

Table 7: 8051 register settings

Register	Value	Meaning
TMOD	21h	Timer 1 in mode 2 (8-bit auto-reload, baud generator); Timer 0 in mode 1 (16-bit)
TH1	FDh	9600 baud at 11.0592 MHz (SMOD = 0)
SCON	50h	Serial mode 1 (8-bit UART, variable baud), receiver enabled (REN = 1)
TH0:TL0	DC00h	Timer 0 overflows every 10 ms; reloaded in the interrupt
TR1, TR0	1	Start both timers
IE (ET0, ES, EA)	1, 1, 1	Enable Timer 0 interrupt, serial interrupt and global interrupts

- **Timer 0 interrupt** reloads TH0/TL0 with DC00h and increments a 16-bit `ticks` counter, giving a 10 ms time base for blinking, beeping and timeouts.

- **Serial interrupt** reads each byte from SBUF and clears RI. If the byte is # it sets a flag. If the next byte is S, I, R or W it stores it as the new status and sets `newcmd`. Any other byte clears the flag. This keeps the interrupt routine very short.
- **Main loop** reads `ticks` safely (Timer 0 interrupt briefly disabled so the two bytes are not read while changing), records the time of the last valid message, and works out the outputs. If more than 5 s (500 ticks) have passed since the last message it signals a lost link. Otherwise it drives the LED and buzzer according to the status. The buzzer sounds only for the first 10 s (1000 ticks) of a new storm warning, so it does not keep sounding.
- Blinking uses the tick count directly: `(t / 50) & 1` toggles every 0.5 s and `(t / 10) & 1` every 0.1 s, so no delay loops are needed and the program never stops responding.

The compiled program occupies only about 0.6 KB of the 8 KB flash memory, leaving plenty of room for additions such as an LCD.

9.5 Flowcharts

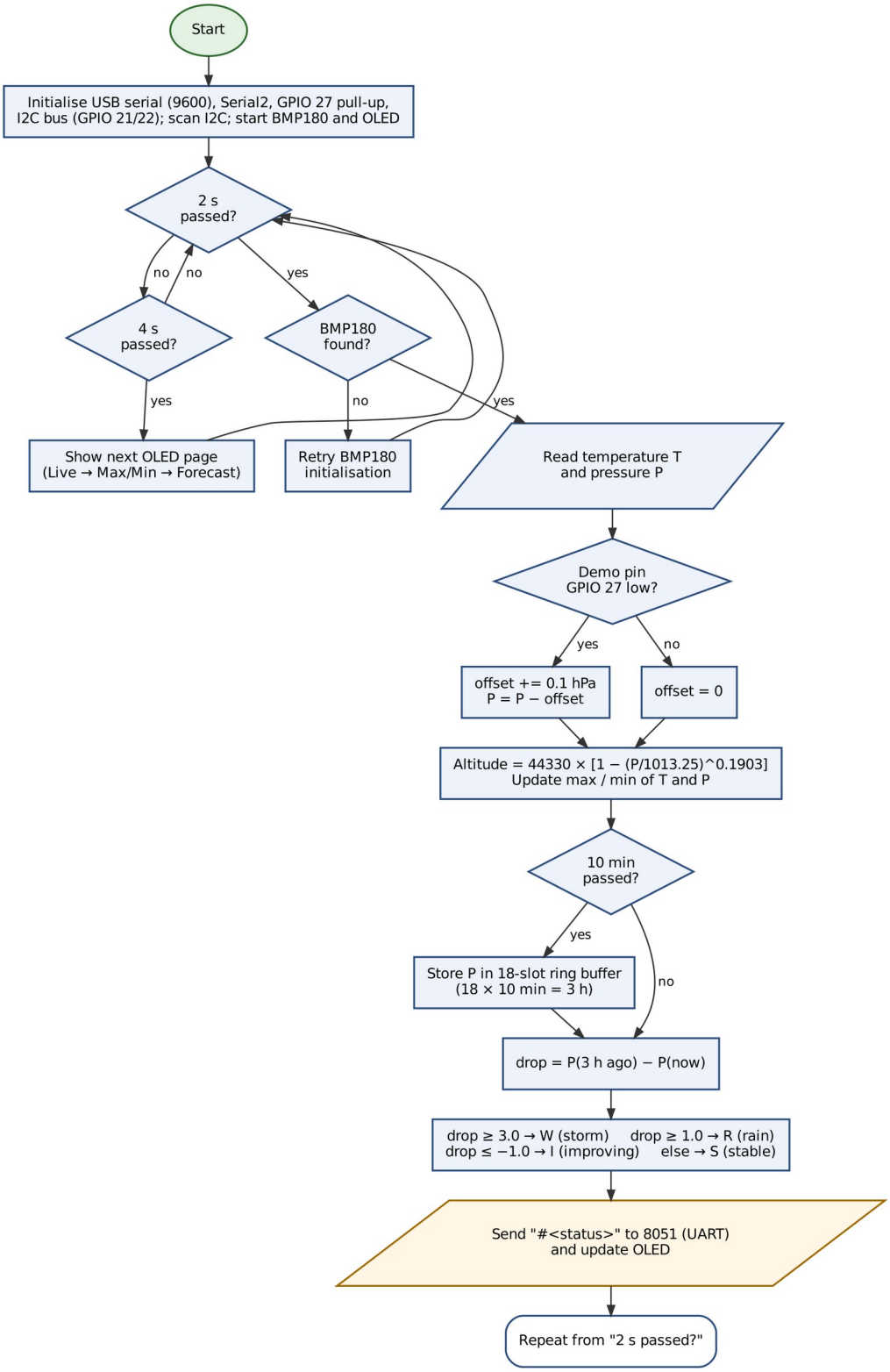


Figure 5: Flowchart of the ESP32 (MYOSA) program

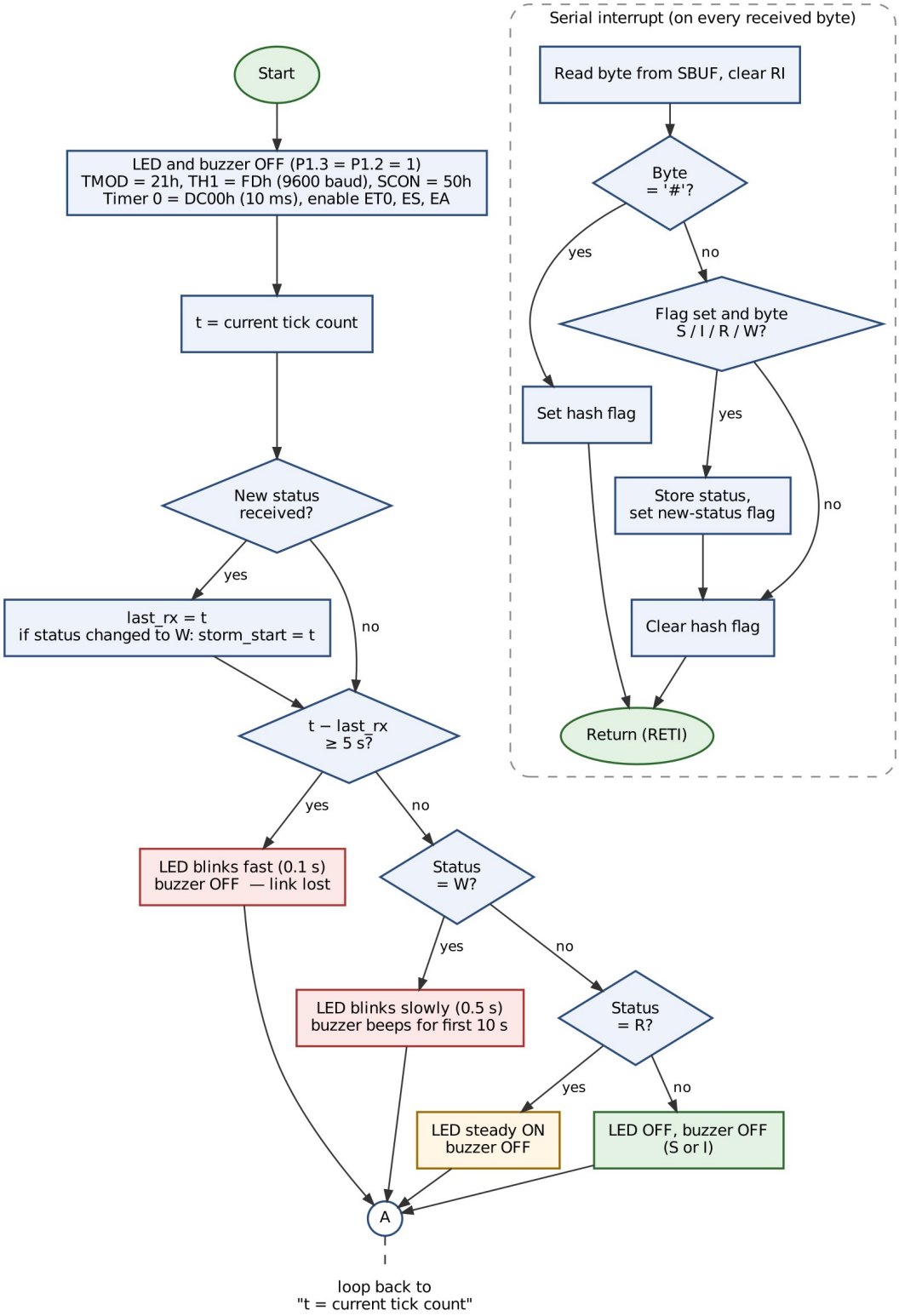


Figure 6: Flowchart of the 8051 program and its serial interrupt routine

10. Algorithm

10.1 ESP32 (sensing unit)

1. Start. Initialise the USB serial port and UART 2 at 9600 baud, the GPIO 27 input with pull-up, and the I2C bus on GPIO 21/22.
2. Scan the I2C bus and initialise the BMP180 sensor and the OLED display.
3. Every 2 seconds: if the BMP180 is not responding, try to initialise it again and skip this cycle.
4. Read the temperature T (°C) and the pressure P (hPa).
5. If GPIO 27 is low (demo mode), increase the offset by 0.1 hPa and subtract it from P; otherwise set the offset to zero.
6. Calculate altitude h = 44330 × [1 − (P/1013.25)^0.1903].
7. On the first reading, fill the 18-slot history with P and set all maximum and minimum values to the current readings. On later readings, update the maximum and minimum of T and P.
8. If 10 minutes have passed since the last sample, store P in the history at position `slot` and advance `slot` (wrapping after 17).
9. Calculate drop = history[slot] − P.
10. If drop ≥ 3.0 the status is W; else if drop ≥ 1.0 the status is R; else if drop ≤ −1.0 the status is I; otherwise the status is S.
11. Send "#" + status + newline to the 8051, print the debug line and redraw the OLED.
12. Every 4 seconds, show the next OLED page (all three pages normally; only Live and Forecast during a warning).
13. Repeat from step 3.

10.2 8051 (alarm unit)

1. Start. Turn the LED and buzzer off (P1.3 = P1.2 = 1).
2. Set TMOD = 21h, TH1 = FDh, SCON = 50h and start Timer 1 (9600 baud).
3. Load Timer 0 with DC00h, start it, and enable the Timer 0, serial and global interrupts.
4. Treat the link as lost until the first message arrives.
5. Read the current tick count t.
6. If a new status has been received, record last_rx = t. If the status has just changed to W, record storm_start = t.
7. If t − last_rx ≥ 500 (5 s), blink the LED every 0.1 s and keep the buzzer off.
8. Otherwise, if the status is W, blink the LED every 0.5 s and sound the buzzer in 0.5 s beeps while t − storm_start < 1000 (10 s).
9. Otherwise, if the status is R, keep the LED on and the buzzer off.
10. Otherwise (S or I) keep the LED and buzzer off.
11. Repeat from step 5.

Serial interrupt routine: read the byte from SBUF and clear RI. If the byte is "#", set the hash flag. Otherwise, if the hash flag is set and the byte is S, I, R or W, store it as the status and set the new-status flag. In all other cases clear the hash flag. Return from the interrupt.

11. Implementation and Simulation Procedure

11.1 Building and uploading the ESP32 program

1. Install VS Code and the PlatformIO IDE extension.
2. Open the project folder (the folder that contains `platformio.ini`). The `platformio.ini` file selects the esp32dev board and the Arduino framework, sets the monitor speed to 9600 and lists the three Adafruit libraries, which PlatformIO downloads automatically.
3. Plug the BMP180 and OLED boards into the MYOSA motherboard and connect it to the laptop by USB.
4. Click Build, then Upload. Open the Serial Monitor and press the EN (reset) button to see the I2C scan and the readings.

11.2 Building the 8051 program

1. Install SDCC 4.6.0 for Windows.
2. Press Ctrl+Shift+B in VS Code. The build task runs `sdcc -mmcs51 -o build/weather_8051.hex weather_8051.c` in the `8051` folder.
3. Because the SDCC folder was not added to the Windows PATH, the full path `C:\Program Files\SDCC\bin\sdcc.exe` is given in `tasks.json`.

11.3 Proteus simulation

1. Create a new Proteus project and place the AT89C52, a red LED with a 330 Ω resistor, a buzzer, POWER terminals and a Virtual Terminal.
2. Wire the LED cathode to P1.3, the buzzer to P1.2 and the Virtual Terminal TXD to P3.0/RXD.
3. In the AT89C52 properties, set Program File to the full path of `weather_8051.hex` and Clock Frequency to 11.0592 MHz. Set the buzzer to 5 V and the terminal to 9600 baud, 8 data bits, no parity, 1 stop bit.
4. Run the simulation and type #R, #W and #S into the terminal to check every alarm state.
5. For the final demonstration, replace the Virtual Terminal with a COMPIM component connected to P3.0/RXD, set its physical port to COM7 and both baud rates to 9600, close the PlatformIO serial monitor (only one program can open the port), and run the simulation with the MYOSA board connected.

12. Testing and Results

12.1 Sensing unit

After uploading, the serial monitor showed the following output (the first line of unreadable characters, printed by the ESP32 boot loader at a different baud rate, is omitted):

```
I2C devices found:
  0x3C
  0x77
BMP180 OK
OLED OK
#S
T=30.6 P=1010.8 A=20 drop=0.0 state=S
#S
T=30.6 P=1010.7 A=21 drop=0.1 state=S
#S
T=30.6 P=1010.6 A=22 drop=0.3 state=S
```

The I2C scan found both devices at their expected addresses. The temperature of 30.6 °C matched the warm room, and the pressure of about 1010.7 hPa is normal for Kochi, which is close to sea level. The altitude estimate of about 20 m is reasonable given that the formula assumes a sea-level pressure of exactly 1013.25 hPa.

12.2 Demonstration mode

With the GPIO 27 jumper fitted, the pressure decreased by about 0.1 hPa per reading and the `drop` value rose steadily. The status changed from S to **R** (RAIN LIKELY on the OLED) when the drop reached 1.0 hPa, and then to **W** (STORM WARNING) at 3.0 hPa. Removing the jumper returned the status to S.

12.3 Alarm unit

The 8051 program was first tested alone in Proteus by typing messages into the Virtual Terminal (Figure 7).

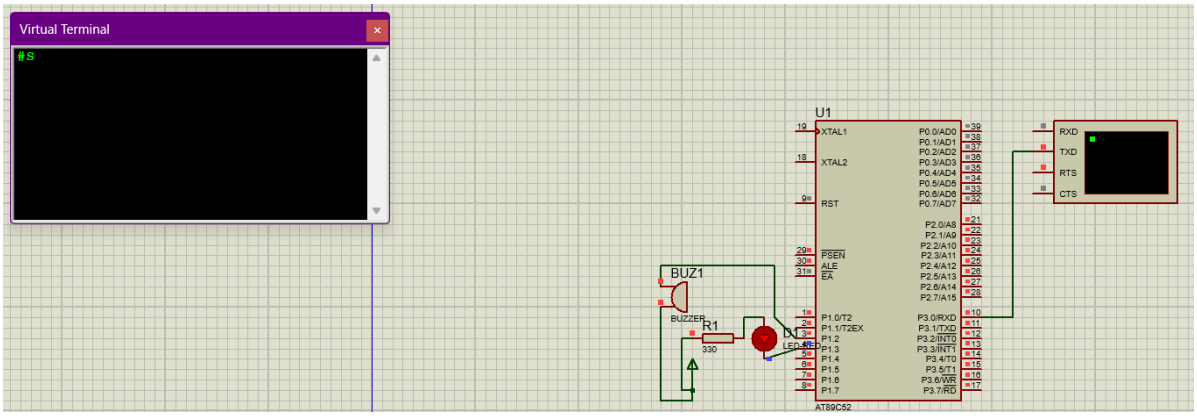


Figure 7: Unit test of the 8051 in Proteus: a status message typed in the Virtual Terminal and the LED responding

12.4 Summary of tests

Table 8: Test cases and results

No.	Test	Expected result	Observed result
1	I2C scan on start-up	0x3C (OLED) and 0x77 (BMP180) found	Both found; "BMP180 OK", "OLED OK"
2	Normal readings	Sensible T, P and altitude every 2 s	T = 30.6 °C, P ≈ 1010.7 hPa, A ≈ 21 m
3	OLED pages	Live, Max/Min, Forecast every 4 s	Pages rotate as expected
4	Demo mode, drop ≥ 1.0 hPa	Status R, OLED "RAIN LIKELY"	Status R shown
5	Demo mode, drop ≥ 3.0 hPa	Status W, OLED "STORM WARNING"	Status W shown
6	8051 with no input	LED blinks fast (link lost)	LED blinks fast
7	Type #R in terminal	LED steady on	LED on for 5 s, then link-lost blink (no repeat sent)
8	Type #W in terminal	LED slow blink, buzzer beeps	As expected
9	Type #S in terminal	LED off	LED off
10	Live link via COMPIM (COM7)	LED/buzzer follow the OLED status	LED and buzzer followed S, R and W
11	USB unplugged during run	LED blinks fast after 5 s	Link-lost indication shown

Test 7 also confirmed that the link-loss timeout works: a single typed message holds for 5 s and then times out, whereas the ESP32 sends a fresh message every 2 s, so the output stays steady in normal operation.

12.5 Problems faced and solutions

Table 9: Problems encountered during development

Problem	Cause	Solution
Humidity read as 0.0 %	The kit has no humidity sensor (the "temp" on the MPU6050 board is only the chip's internal temperature)	Humidity removed; temperature taken from the BMP180
"sdcc is not recognized"	SDCC folder not on the Windows PATH	Full path to sdcc.exe written in tasks.json
Proteus: "Unable to open HEX file"	Project saved inside the .vscode folder, so the relative path was wrong	Project saved in the main folder and the full HEX path entered
8051 received nothing	Terminal TXD was wired to 8051 TXD (pin 11) instead of RXD (pin 10)	Rewired TXD → P3.0/RXD
Risk of garbled serial data	Proteus default clock is 12 MHz (8.5 % baud error)	Clock set to 11.0592 MHz
Status only held for 5 s when typed	Intended link-loss timeout	Normal; the ESP32 repeats the status every 2 s
COM port busy	Only one program can open COM7	Close the serial monitor before running Proteus

13. Cost Estimation

The MYOSA kit was already available to the team, and the 8051 part was simulated, so very little money was actually spent. The table below gives approximate local market prices for the equivalent parts, and also the extra parts needed to build the 8051 alarm unit as real hardware.

Table 10: Cost estimation (approximate prices, 2026)

Item	Qty	Approx. cost (₹)
A. Sensing unit (MYOSA kit or equivalent parts)		
ESP32 development board (MYOSA motherboard equivalent)	1	450
BMP180 pressure sensor module	1	150
0.96" SSD1306 OLED display (I2C)	1	250
USB cable and jumper wires	1 set	150
Subtotal A		**1000**
B. 8051 alarm unit as real hardware		
AT89S52 microcontroller with 40-pin IC base	1	90
11.0592 MHz crystal and 2 × 22 pF capacitors	1 set	25
10 µF capacitor, 10 kΩ, 330 Ω, 1 kΩ resistors, red LED	1 set	15
5 V buzzer and BC557 transistor	1 each	30
Breadboard	1	90
USBasp programmer (for AT89S52)	1	250
Subtotal B		**500**
C. Software (VS Code, PlatformIO, SDCC: free; Proteus: college licence)		0
Total (A + B + C)		**≈ 1500**

Since the project was built with an existing kit and a simulated 8051, the actual expenditure was negligible. A complete stand-alone hardware version would cost about ₹1500.

14. Applications

- **Farms and gardens:** advance warning to cover crops, delay spraying or harvest before rain.
- **Homes and schools:** a simple local weather display with rain and storm alerts.
- **Fishing communities and boat users:** an extra warning of rapidly falling pressure before going out.
- **Construction sites and outdoor events:** deciding when to protect materials or move activities indoors.
- **Altitude indication:** relative height changes for trekking or drone experiments (with suitable calibration).
- **Education:** demonstrating I2C, UART, timers, interrupts and inter-processor communication in embedded systems laboratories.

15. Merits and Demerits

15.1 Merits

- Low cost: about ₹1000 for the sensing unit, built from widely available parts.
- Gives an actual **forecast** from the pressure trend, not just a reading of current conditions.
- Clear warnings: OLED text, a steady or blinking LED and a buzzer, so the alert is noticed even from a distance.
- Fault awareness: the 8051 detects a broken link within 5 s and shows it, instead of silently showing old information.
- Robust protocol: the "#" marker lets the 8051 ignore debug text and noise on the line.
- Sensible division of work: the ESP32 handles floating-point maths, I2C and graphics; the 8051 handles timing-critical outputs with interrupts.
- Built-in demonstration mode allows the full behaviour to be shown in about a minute.
- Easy to extend: the ESP32 has Wi-Fi and Bluetooth, and the 8051 still has more than 7 KB of free program memory.

15.2 Demerits

- No humidity, rainfall or wind measurement, so the forecast is based on pressure alone and is only a simple indication.
- The trend needs about three hours after switch-on before it reflects real weather changes, because the history starts filled with the first reading.
- The altitude is only an estimate, because it assumes a fixed sea-level pressure of 1013.25 hPa.
- Indoors the BMP180 temperature can read higher than the outside air because of nearby electronics and room heat.
- The 8051 part was simulated; a laptop running Proteus is needed for the complete demonstration.
- Communication is one-way: the ESP32 does not know whether the 8051 received the message.
- Data is not stored, so readings are lost when power is switched off.

16. Future Scope

- **Build the 8051 alarm unit in hardware** on a breadboard or PCB and connect it directly to GPIO 25 (UART 2), as the code already supports, and add a 16×2 LCD to the 8051 to show the status.
- **Add a humidity sensor** (for example SHT31 or AHT20) to calculate relative humidity, dew point and heat index.

- **Better forecasting** using the Zambretti forecaster, which combines pressure, pressure trend, season and wind direction, together with sea-level pressure correction using the known station altitude.
- **Wi-Fi dashboard:** use the ESP32's Wi-Fi to upload readings to a cloud platform or a web page and send phone notifications when a storm warning occurs.
- **Data logging** on an SD card with a real-time clock, so that the pressure history survives power cuts and long-term graphs can be drawn.
- **Additional sensors** such as rain detection, light intensity, air quality and wind speed for a more complete station.
- **Two-way communication** with acknowledgements and a checksum, so that each side can confirm correct delivery.
- **Battery or solar power** with low-power sleep modes for outdoor installation.

17. Conclusion

We designed, built and tested a Mini Weather Station that measures temperature and atmospheric pressure, estimates altitude, keeps maximum and minimum values and, most importantly, predicts rain and storms from the three-hour pressure tendency. The ESP32 on the MYOSA kit handles the sensing, calculations and OLED display, while the 8051 (AT89C52) acts as a dedicated alarm unit that receives the weather status over a 9600-baud serial link and drives a warning LED and buzzer. It can also detect a broken link.

All the tests gave the expected results. The sensors were detected correctly, the readings were realistic, the demonstration mode produced rain and storm warnings, and the 8051 responded correctly both to hand-typed messages and to live data from the hardware through the Proteus COMPIM link. During development we solved practical problems with tool paths, simulation settings, UART wiring and baud-rate accuracy. These showed us why details such as the 11.0592 MHz crystal and the TXD-to-RXD connection matter.

The project gave us hands-on experience with I2C sensor interfacing, UART communication between two microcontroller families, 8051 timer and interrupt programming, simple real-time data processing and circuit simulation. It provides a solid base that can be extended into a complete, connected weather monitoring system.

References

1. M. A. Mazidi, J. G. Mazidi and R. D. McKinlay, *The 8051 Microcontroller and Embedded Systems: Using Assembly and C*, 2nd ed., Pearson Education.
2. Atmel Corporation, *AT89C52 8-bit Microcontroller with 8K Bytes Flash*, datasheet.
3. Bosch Sensortec, *BMP180 Digital Pressure Sensor*, datasheet BST-BMP180-DS000.
4. Solomon Systech, *SSD1306 128 × 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*, datasheet.
5. Espressif Systems, *ESP32 Series Datasheet* and *ESP32 Technical Reference Manual*.
6. NXP Semiconductors, *UM10204 I2C-bus Specification and User Manual*.
7. IEEE Sensors Council, *MYOSA – Make Your Own Sensors Applications*, ieee-sensors.org/myosa.
8. SDCC – Small Device C Compiler, *SDCC Compiler User Guide*, sdcc.sourceforge.net.
9. PlatformIO Labs, *PlatformIO Documentation – Espressif 32 platform*, docs.platformio.org.
10. Adafruit Industries, *Adafruit BMP085 Library*, *Adafruit SSD1306* and *Adafruit GFX Library* (GitHub).
11. Labcenter Electronics, *Proteus Design Suite 8 – ISIS User Manual* (COMPIM and Virtual Terminal models).
12. Met Office (UK), *National Meteorological Library Fact Sheets* – pressure and pressure tendency terms.

Appendix A – ESP32 Program (src/main.cpp)

```
/*  ====================================================================
    MINI WEATHER STATION  -  ESP32 side (MYOSA motherboard)
    Tool : VS Code + PlatformIO (libraries are listed in platformio.ini
           and download automatically on the first build)

    The ESP32 does the main work:
      - reads the BMP180 (temperature + pressure, altitude from pressure)
      - keeps max / min of temperature and pressure
      - watches how fast pressure falls over 3 hours:
            fall >= 1.0 hPa  -> RAIN LIKELY
            fall >= 3.0 hPa  -> STORM WARNING
            rise >= 1.0 hPa  -> IMPROVING
      - shows everything on the MYOSA OLED (pages change every 4 s)
      - sends a status to the 8051 every 2 s (9600 baud) as "#S", "#I",
        "#R" or "#W"  ('#' marks the start, then the letter):
            'S' stable   'I' improving   'R' rain likely   'W' storm warning
        The 8051 drives the warning LED and the buzzer.

    SIMULATED 8051 (Proteus): the status goes out on the USB cable
    (Serial, 9600 baud) -> laptop COM port -> Proteus COMPIM -> 8051 RXD.
    Close the PlatformIO Serial Monitor while Proteus is running, because
    only one program can use the COM port at a time.
    The same status also goes out on GPIO 25 (Serial2) for a real 8051.

    DEMO MODE: connect GPIO 27 to GND. Pressure is made to fall by
    0.1 hPa every 2 s, so "RAIN LIKELY" comes after ~20 s and
    "STORM WARNING" after ~60 s. Remove the wire for real data.
    ==================================================================== */

#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_BMP085.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

/* ---------- pins ---------- */
#define SDA_PIN     21
#define SCL_PIN     22
#define TX_TO_8051  25      // -> 8051 P3.0 (RXD)
#define RX_UNUSED   26      // not used
#define DEMO_PIN    27      // tie to GND for demo mode

/* ---------- I2C addresses ---------- */
#define OLED_ADDR   0x3C    // MYOSA OLED (SSD1306)

/* ---------- settings ---------- */
#define SLOTS       18            // 18 x 10 min = 3 hour window
#define SLOT_MS     600000UL      // 10 minutes
#define RAIN_DROP   1.0           // hPa
#define STORM_DROP  3.0           // hPa
#define READ_MS     2000UL        // read sensors every 2 s
#define PAGE_MS     4000UL        // change OLED page every 4 s

Adafruit_BMP085  bmp;
Adafruit_SSD1306 oled(128, 64, &Wire, -1);

bool bmpOK = false, oledOK = false, firstRead = true;
float t, p, a;
float tmax, tmin, pmax, pmin;
float hist[SLOTS];
int   slot = 0;
float drop = 0;                 // old - now  (+ve = pressure falling)
char  state = 'S';
int   page = 0;
float demoOffset = 0;

void i2cScan() {
  Serial.println("I2C devices found:");
  for (uint8_t addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) Serial.printf("  0x%02X\n", addr);
  }
}

/* ---------- read sensors + weather logic ---------- */
void updateWeather() {
  t = bmp.readTemperature();
  p = bmp.readPressure() / 100.0;           // Pa -> hPa
```

```
  if (digitalRead(DEMO_PIN) == LOW) demoOffset += 0.1;   // simulate falling pressure
  else                              demoOffset = 0;
  p -= demoOffset;

  a = 44330.0 * (1.0 - pow(p / 1013.25, 0.1903));        // altitude estimate

  if (firstRead) {                          // first reading: fill history, set max/min
    for (int i = 0; i < SLOTS; i++) hist[i] = p;
    tmax = tmin = t;  pmax = pmin = p;
    firstRead = false;
  }

  tmax = max(tmax, t);   tmin = min(tmin, t);
  pmax = max(pmax, p);   pmin = min(pmin, p);

  static uint32_t lastSlot = millis();
  if (millis() - lastSlot >= SLOT_MS) {      // store one sample every 10 min
    lastSlot = millis();
    hist[slot] = p;
    slot = (slot + 1) % SLOTS;
  }

  drop = hist[slot] - p;                    // hist[slot] = oldest sample (~3 h ago)

  char old = state;
  if      (drop >= STORM_DROP) state = 'W';
  else if (drop >= RAIN_DROP)  state = 'R';
  else if (drop <= -RAIN_DROP) state = 'I';
  else                         state = 'S';
  if (state != old && (state == 'R' || state == 'W')) page = 2;   // jump to forecast
}

const char* stateText() {
  switch (state) {
    case 'W': return "STORM WARNING";
    case 'R': return "RAIN LIKELY";
    case 'I': return "IMPROVING";
    default:  return "STABLE";
  }
}

/* ---------- OLED pages ---------- */
void drawPage() {
  if (!oledOK) return;
  oled.clearDisplay();
  oled.setTextColor(SSD1306_WHITE);
  oled.setTextSize(1);
  oled.setCursor(0, 0);

  if (page == 0) {                          // live readings
    oled.println("WEATHER  - LIVE");
    oled.println();
    oled.printf("Temp : %.1f C\n\n", t);
    oled.printf("Press: %.1f hPa\n\n", p);
    oled.printf("Alt  : %.0f m\n", a);
  }
  else if (page == 1) {                     // max / min
    oled.println("MAX / MIN");
    oled.println();
    oled.printf("Temp  max %.1f C\n", tmax);
    oled.printf("      min %.1f C\n\n", tmin);
    oled.printf("Press max %.1f\n", pmax);
    oled.printf("      min %.1f hPa\n", pmin);
  }
  else {                                    // forecast
    oled.println("FORECAST");
    oled.println();
    oled.printf("dP (3h): %+.1f hPa\n", -drop);
    oled.println();
    oled.setTextSize(state == 'W' ? 1 : 2);
    oled.println(stateText());
  }
  oled.display();
}

/* ---------- setup / loop ---------- */
void setup() {
  Serial.begin(9600);                       // USB -> PC -> Proteus 8051
  Serial2.begin(9600, SERIAL_8N1, RX_UNUSED, TX_TO_8051);
  pinMode(DEMO_PIN, INPUT_PULLUP);
  Wire.begin(SDA_PIN, SCL_PIN);
```

```
  i2cScan();                                  // expect 0x3C (OLED) and 0x77 (BMP180)
  bmpOK  = bmp.begin();
  oledOK = oled.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
  Serial.println(bmpOK  ? "BMP180 OK" : "BMP180 NOT FOUND");
  Serial.println(oledOK ? "OLED OK"   : "OLED NOT FOUND");

  if (oledOK) {
    oled.clearDisplay();
    oled.setTextColor(SSD1306_WHITE);
    oled.setTextSize(1);
    oled.setCursor(0, 0);
    oled.println("Weather Station");
    oled.println("Starting...");
    oled.display();
  }
}

void loop() {
  static uint32_t lastRead = 0, lastPage = 0;

  if (millis() - lastRead >= READ_MS) {
    lastRead = millis();
    if (!bmpOK) { bmpOK = bmp.begin(); return; }   // no data -> 8051 blinks LED fast

    updateWeather();
    Serial.printf("#%c\n", state);          // to simulated 8051 (via USB)
    Serial2.printf("#%c\n", state);         // to a real 8051 on GPIO 25
    Serial.printf("T=%.1f P=%.1f A=%.0f drop=%.1f state=%c\n",
                  t, p, a, drop, state);
    drawPage();
  }

  if (!firstRead && millis() - lastPage >= PAGE_MS) {
    lastPage = millis();
    if (state == 'R' || state == 'W') page = (page == 2) ? 0 : 2;  // live <-> forecast
    else                              page = (page + 1) % 3;
    drawPage();
  }
}
```

platformio.ini

```
; ESP32 side of the weather station (MYOSA motherboard)
; Build / Upload / Monitor with the PlatformIO buttons on the bottom bar.

[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 9600
lib_deps =
    adafruit/Adafruit BMP085 Library
    adafruit/Adafruit SSD1306
    adafruit/Adafruit GFX Library
```

Appendix B – 8051 Program (8051/weather_8051.c)

```
/*  ====================================================================
    MINI WEATHER STATION  -  8051 side (ALARM UNIT)
    Board  : AT89S52 (any 8052) @ 11.0592 MHz crystal
    Tool   : SDCC compiler (run from VS Code: Ctrl+Shift+B)
    Output : 8051/build/weather_8051.hex  -> load into Proteus AT89C52

    The ESP32 sends a status every 2 s on RXD (9600 baud): '#' then a letter
        'S' = stable      'I' = improving
        'R' = rain likely 'W' = storm warning

    The 8051 drives the alarm:
        S / I  -> LED off, buzzer off
        R      -> LED on (steady)
        W      -> LED blinks, buzzer beeps for 10 s then stops
        no letter for 5 s (ESP32 off / wire loose) -> LED blinks fast
    ==================================================================== */

#include <8051.h>

#define BUZ P1_2            /* buzzer (active-LOW) */
#define LED P1_3            /* LED cathode to pin, anode via resistor to VCC */

#define ON   0              /* active-LOW outputs */
#define OFF  1

#define BEEP_TIME  1000     /* 10 s   (in 10 ms ticks) */
#define NO_DATA    500      /* 5 s    */

volatile unsigned int  ticks = 0;     /* +1 every 10 ms */
volatile unsigned char cmd = 'S';     /* last status letter */
volatile unsigned char newcmd = 0;
volatile unsigned char hash_seen = 0; /* '#' received, letter comes next */

/* Timer0: 10 ms tick */
void timer0_isr(void) __interrupt(1)
{
    TH0 = 0xDC; TL0 = 0x00;           /* 65536 - 9216 = 0xDC00 */
    ticks++;
}

/* Serial: receive status letter */
void serial_isr(void) __interrupt(4)
{
    unsigned char c;
    if (RI) {
        RI = 0;
        c = SBUF;
        if (c == '#') {
            hash_seen = 1;                  /* start of a status */
        } else {
            if (hash_seen && (c == 'S' || c == 'I' || c == 'R' || c == 'W')) {
                cmd = c;
                newcmd = 1;
            }
            hash_seen = 0;                  /* ignore any other text */
        }
    }
    if (TI) TI = 0;
}

unsigned int now(void)                /* safe read of 16-bit tick count */
{
    unsigned int t;
    ET0 = 0; t = ticks; ET0 = 1;
    return t;
}

void main(void)
{
    unsigned int t0, last_rx = 0, storm_start = 0;
    unsigned char prev = 'S', nodata;

    BUZ = OFF; LED = OFF;

    TMOD = 0x21;          /* Timer1 mode 2 (baud), Timer0 mode 1 (tick) */
    TH1  = 0xFD;          /* 9600 baud @ 11.0592 MHz */
    SCON = 0x50;          /* serial mode 1, receive on */
    TR1  = 1;
    TH0  = 0xDC; TL0 = 0x00; TR0 = 1;
```

```
    ET0  = 1; ES = 1; EA = 1;

    last_rx = now() - NO_DATA;        /* start in "no data" until ESP32 talks */

    while (1) {
        t0 = now();

        if (newcmd) {                 /* new letter arrived */
            newcmd = 0;
            last_rx = t0;
            if (cmd == 'W' && prev != 'W') storm_start = t0;   /* storm just started */
            prev = cmd;
        }

        nodata = ((t0 - last_rx) >= NO_DATA);

        /* LED */
        if (nodata)          LED = ((t0 / 10) & 1) ? ON : OFF;  /* fast blink: 0.1 s */
        else if (cmd == 'W') LED = ((t0 / 50) & 1) ? ON : OFF;  /* slow blink: 0.5 s */
        else if (cmd == 'R') LED = ON;
        else                 LED = OFF;

        /* buzzer: beeps for 10 s when storm starts */
        BUZ = (!nodata && cmd == 'W' && (t0 - storm_start) < BEEP_TIME
               && ((t0 / 50) & 1)) ? ON : OFF;
    }
}
```

.vscode/tasks.json (8051 build task)

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Build 8051 (SDCC)",
      "type": "process",
      "command": "C:\\Program Files\\SDCC\\bin\\sdcc.exe",
      "args": ["-mmcs51", "-o", "build/weather_8051.hex", "weather_8051.c"],
      "options": { "cwd": "${workspaceFolder}/8051" },
      "group": { "kind": "build", "isDefault": true },
      "problemMatcher": []
    }
  ]
}
```